- To create something from a file

  kubectl create -f <file path> [it'll create if resource doesn't exist]

  kubectl apply -f <file path> [create if doesn't exist; update if exists]

- To edit any resource live

  kubectl edit <resource type> <resource name>

  kubectl edit pod my-pod

- To see all the details of a resource along with events

  kubectl describe <resource type> <resource name>

  kubectl describe pod my-pod

- To see *live resource configuration* in *yaml* format

  kubectl get <resource type> <resource name> -o yaml

  kubectl get pod my-pod -o yaml

- Too see all the things of the available resources

  kubectl get <resource type> <resource name> -o wide

- To get a yaml configuration file of any resource to start with,

  --dry-run=client -o yaml append this in the imperative commands.

  kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml

- Pod
  - To create a pod ("v1")

    kubectl run <pod name> --image=<image name>

    kubectl run my-pod --image=nginx
  - Whatever ports you write inside containers (inside pods) are not exposed, that is just used for user's docs or to search pods a/c to ports. The traffic reaches to a specific port of a container with Service only.
  - If a pod contains multiple containers,

    each container: *same IP, different ports* [because IP is assigned to Pod]

- ReplicaSet
  - ReplicaSet *("apps/v1")* and ReplicationController *("v1")* cannot be created with imperative commands.
  - ReplicaSet contains spec.*selector* field, which ReplicationController doesn't have.
  - *Selector* contains *matchLabels* or *matchExpressions*.

- *template* (pod) should contain same or more labels present in *selector*. Selector's labels work with AND rule, not OR. All the labels must be present in the pods to be selected.
- To scale replica set:

  kubectl scale --replicas=<replica count**> replicaset <**replicaset name**>**

  kubectl scale --replicas=4 replicaset my-rs

- Deployment
  - Deployment can be created with Imperative way because Kubernetes auto-fill the required complex fields like template, labels, selectors.

    kubectl create deployment <deployment name**> --image=<**image name**> --replicas=<**replica count**>**
  - It provide *pull, upgrade, rolling update (or) recreate, rollback (undo), pause (rollout pause), resume (rollout resume)* over ReplicaSet.
  - It create new versions, and *scales down* old ReplicaSet gradually and *scales up* new ReplicaSet gradually.
  - *maxUnavailable* (maximum allowed unavailable pods. Percentage or count), *maxSurge* (maximum allowed available pods. Percentage or count)

    [scale up of new pods & scale down of old pods happens w.r.t. *maxUnavailable* & *maxSurge*]
  - To see *rollout status*,

    kubectl rollout status deployment <deployment name**>**

    To see *rollout history*,

    kubectl rollout history deployment <deployment name**>**

    [for this --record has to be provided; but its deprecated]

    To *undo* the deployment

    kubectl rollout undo deployment <deployment name**>**
  - Deployment = ReplicaSet + [rollout, rollback, versioning(revisions), rollout strategy (Recreate, RollingUpdate), Pause, Resume]
- Deployment Strategy
  - RollingUpdate: gradually remove old pods and create new pods.
  - Recreate: remove all old pods and create new pods.
- Service

- Required to communicate with the pods.

  [Pod's IP can be used; but shouldn't because Pods might get re-created]
- External to Pod: NodePort

  Pod to Pod: ClusterIP (NodePort can also be used; but not preferred)

  Load Balancing: LoadBalancer [works in cloud based clusters]
- type:NodePort

  ports: [{ nodePort: x, port: y, targetPort: z }, { .. }, ..]

  port means service's port

  in here, whatever port you assign for *service*, it doesn't matter because it is there only to connect nodePort to targetPort (pod's port).
- type: ClusterIP

  ports: [{ port: x, targetPort: y }, { .. }, ..]
- For inter-pod communication

  <service name>:<service port>

  service -> name, selector; pod: labels; <-- these are important
- *selector* doesn't contain *matchLabels, matchExpressions*.

  Direct *key:value* pairs (labels) to select the target pods.
- Service's redirection to Pod is cluster level. So service's and pod's host nodes might be different.

- **kubeadm** tool can be used to create a cluster and configure multiple nodes (control plane & worker nodes). it generates the required certificates and all.

  https://github.com/kodekloudhub/certified-kubernetes-administrator-course/tree/master/kubeadm-clusters/generic
- **cgroup** (Control Group)
  - Used to enforce limits by Kernel.
  - **mount | grep cgroup** : to see the mounted path (/sys/fs/cgroup)

    cgroup2 for modern systems; cgroup for older systems;
  - Files & Directories inside *cgroup* are handled by ?
    - all files : *Kernel*
    - directories of format *cgroup.*, cpu.*, memory.*, io.** : *Kernel*
    - directories of format *.slice, .scope, .service, .mount* : ***systemd***
    - other directories (*docker, kubepods ..etc*): respective owners

- *system.slice* contains *containerd.service*, it means container runtime (containerd) is managed by *systemd* itself.
  In case of separately handled, both will not get to know each other's usage and request for more resources; at the end kernel will take action and stop some resources which might be important.
- Owners writes into the respective files, Kernel enforces limits.

- Internals of *kubectl apply*
  - 3 configs are there:
    config file (static yaml configuration file used to create a resource)
    live config file (configuration file of the running resource; *kubectl get <resource type> <resource name> -o yaml*)
    last applied config (latest config applied in *declarative* way) [its in JSON format]
  - Running resource updated with:
    - imperative command (or) *kubectl edit* command: live config is updated, last applied config will remain same
    - Declarative way (*kubectl apply*) : live applied config will be updated, config file will be updated of-course.
  - Configs updation:

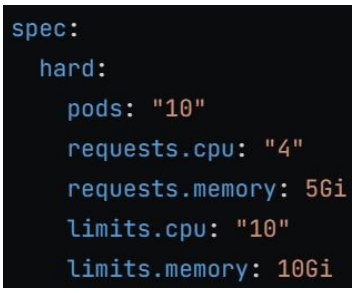
    If a field is there in *live config* (ex: an extra level was added using *kubectl edit* command) but not in both *config file* and *last applied config* then field will remain as it is.

    If a field is there in either *config file* or *last applied config* or *both* then that field will be updated (doesn't matter if that field is there in *live config* or not)

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

- namespace
  - Namespace defines a logical boundary where resources can be present.

- Same named resources cannot stay inside a single namespace; but can be present in different namespaces.
- Ex: dev, prod, ..etc namespaces can be created.
- To query/update resources in a specific namespace, use
  -n <namespace name> (or) **--namespace** <namespace name>
  kubectl get pods -n dev-namespace
- In the yaml, *metadata.namespace* is the required field to set the namespace.
- Clustered configured with *kubeadm*, contains *kube-system* namespace which contains control plane resources.
- To access a service of another namespace
  <service name>.<target namespace name>**.svc.cluster.local**
  db-service.dev-namespace.svc.cluster.local

- ResourceQuota
  - Used to limit resources for ***namespace*** level.
  - 

-
- F
- F
- F
- F
- F
- F
- F
- F
- F
- F

々